

CTRL + ALT + ESCAPE

Master Challenge Design

Official Challenge and Answer Document. ORGANIZERS ONLY: this file contains every solution. Format: 6 levels, 120 minutes, teams of 3-4. All answers in this document were verified by script before release.

Event Narrative

NEXUS is an experimental AI system that monitors and defends a university research network. At 02:14 UTC an intruder logged in with stolen credentials, exported NEXUS's model weights and planted imposter processes in its memory. NEXUS's safeguards fired: every core module locked itself, and NEXUS split its Master Recovery Key across six security layers so that no single compromised component could ever surrender it.

Each team is an emergency response unit. Every layer holds one failed subsystem and one key fragment. The layers must be opened in order, because each layer's output is evidence for the next. Solve a layer, recover its fragment, and NEXUS restores a little more of itself. Fail to clear a layer inside its time window and the lockdown becomes permanent for that team.

The levels follow the order in which NEXUS failed: its identity was scrambled (Level 1), its guard models were misjudged (Level 2), imposters hid in its memory (Level 3), the intruder's path through authentication and the firewall must be traced (Level 4), its scheduler was found corrupt (Level 5), and finally everything recovered is combined to rebuild the Master Access Key (Level 6).

Challenge Progression

Level	Theme	Difficulty	Time	Primary Skill	Output
1	Corrupted Identifier	Easy-ish	15 min	Decoding: hex, ASCII, binary, rotation cipher	Word (8 letters)
2	The Two Guards	Medium	18 min	ML evaluation: confusion matrix, precision, recall, F1	Letter + number
3	The Imposter Processes	Medium	22 min	Data analysis: discovering a hidden rule in a table	Number
4	The Ghost in the Auth Log	Medium/Hard	22 min	Security: log analysis, firewall rule order, hex	Hex + number

Level	Theme	Difficulty	Time	Primary Skill	Output
5	The Memory Walker	Hard	23 min	Code tracing and debugging	Two numbers
6	Master Recovery	Hard / Final Boss	20 min	Synthesis: cipher + repaired code + fragments	Master Access Key

Total: 120 minutes.

Organizer Ground Rules

- **Allowed:** pen, paper, basic calculator. **Not allowed:** internet, AI tools, and running code on any device. Levels 5 and 6 are designed to be traced by hand; if teams can execute code, Level 5 loses most of its difficulty.
- **Submissions:** free text, case-insensitive. Ignore spaces, hyphens and commas unless a level says otherwise. Maximum 5 wrong submissions per level, then a 3-minute cooldown (stops brute-force guessing).
- **Timers:** a level's timer starts when its text is revealed. Unused time does not carry over. A team with no correct submission when the timer ends is eliminated.
- **Hints:** organizers reveal hints manually, Hint 1 before Hint 2. Suggested penalty: Hint 1 = -5 points, Hint 2 = -10 points (out of 100 per level).
- **Fragments:** every level awards one fragment. Teams must keep all of them; Level 6 cannot be solved without them.

LEVEL 1 — Corrupted Identifier

Difficulty

Easy-ish

Recommended Time

15 minutes

Technical Domain

Decoding / Cryptography (hexadecimal, ASCII, binary, rotation cipher)

Story Context

NEXUS's identity module, the component that proves to the other five layers that NEXUS is NEXUS, was scrambled by the intruder. Just before it crashed, it dumped what remained into an emergency buffer. Until the identifier is recovered, no layer will talk to any other.

Challenge Presented to Participants

```
>> IDENTITY MODULE: FAULT 0x01
>> The system cannot recognise its own name.
>> Recovery note found in the buffer:
    1. The identifier was stored as ASCII text, written as hexadecimal bytes.
    2. Before storage, every letter was ROTATED: moved FORWARD through
       the alphabet by N places (after Z comes A again).
    3. N is the number of 1-bits in the status byte 0xB6
       (count them in its binary form).
>> Recover the original identifier.
```

Data / Code / Clues Given to Participants

```
EMERGENCY BUFFER : 58 4A 53 59 4E 53 4A 51
STATUS BYTE      : 0xB6
ASCII REFERENCE  : 0x41 = A, 0x42 = B, 0x43 = C ...
                  letters continue in order up to 0x5A = Z
```

Expected Participant Reasoning

Teams must notice that two independent transformations were applied and that one of them (the rotation amount) is not given directly: it must be derived from the status byte. A student comfortable with hex handles the first step, another recognises a Caesar-style rotation, and a third converts 0xB6 to binary. The result must be a real word, which acts as a built-in check.

Step-by-Step Solution

- 1. Hex to ASCII.** 0x41 is A, so a byte's letter is its value minus 0x41, counted from A. For example $0x58 - 0x41 = 0x17 = 23$, which is the 24th letter, X. 58 = X, 4A = J, 53 = S, 59 = Y, 4E = N, 53 = S, 4A = J, 51 = Q. Intermediate string: **XJSYNSJQ** (not a word, so a rotation is hiding it).
- 2. Find N.** $0xB6 = 1011\ 0110$ in binary. Count the 1-bits: $1+0+1+1+0+1+1+0 = 5$. So $N = 5$.
- 3. Undo the rotation.** Move every letter BACK 5 places: X to S, J to E, S to N, Y to T, N to I, S to N, J to E, Q to L.

4. Result: **SENTINEL**, an eight-letter English word.

Final Answer

SENTINEL

Key Fragment

SENTINEL (teams must remember this word: it is the identity NEXUS booted with.)

Concepts Tested

Hexadecimal to ASCII, hex to binary conversion, bit counting, Caesar rotation and its inverse, sanity-checking a decode against a real word.

Common Mistakes

- Rotating FORWARD again instead of undoing it: gives COXDSXOV.
- Counting the 0-bits (three) instead of the 1-bits: gives UGPVKPGN.
- Treating 0xB6 as decimal 182, or using 0xB6 mod 26 as the shift.
- Reading the hex bytes as decimal numbers.
- Submitting the intermediate string XJSYNSJQ.

Hint 1

The buffer is not the identifier yet. Two different transformations were applied, and one of them can only be undone using a different clue on the screen.

Hint 2

Convert the hex bytes to letters first (0x41 is A); you will get a string that is not a word. Then write 0xB6 as 1011 0110 and count the ones to learn how far each letter must travel back.

Evaluation Criteria

Accepted answer: **SENTINEL** (any capitalisation, no spaces).

Not accepted: XJSYNSJQ (intermediate), COXDSXOV, UGPVKPGN, or any other string. If a team submits XJSYNSJQ, the organizer may say only: 'That is not the identifier yet.'

LEVEL 2 — The Two Guards

Difficulty

Medium

Recommended Time

18 minutes

Technical Domain

AI / Machine Learning: classification, confusion matrix, evaluation metrics

Story Context

With its identity restored, NEXUS reboots its threat-detection layer and finds two classifier models, Guard-A and Guard-B. It must hand security to exactly one of them. NEXUS proudly reports that Guard-B scored the best accuracy, but NEXUS's judgment is exactly what the intruder tampered with. The team must decide which guard is actually trustworthy.

Challenge Presented to Participants

```
>> THREAT DETECTION: OFFLINE
>> Two guard models are available. Only ONE may be activated.
>> NEXUS self-report: 'Guard-B is my strongest model: 98% accuracy.'
>> Both guards were tested on the same 1,000 network events.
    20 were real ATTACKS. 980 were NORMAL traffic.
>> Missing a real attack is catastrophic. A false alarm is only an annoyance.
>> 1. Decide which guard NEXUS should trust.
>> 2. Compute that guard's F1 score as a WHOLE-NUMBER PERCENTAGE
    (round to the nearest integer).
>> Authorisation format: [GUARD LETTER]-[F1 PERCENT]    (format example: C-
72)
```

Data / Code / Clues Given to Participants

For this task, 'positive' means ATTACK.

Guard-A

	Actual ATTACK	Actual NORMAL
Predicted ATTACK	16	24
Predicted NORMAL	4	956

Guard-B

	Actual ATTACK	Actual NORMAL
Predicted ATTACK	0	0
Predicted NORMAL	20	980

TP = predicted ATTACK and actually ATTACK FP = predicted ATTACK but actually NORMAL
 FN = predicted NORMAL but actually ATTACK TN = predicted NORMAL and actually NORMAL

Accuracy = $(TP + TN) / \text{total events}$
 Precision = $TP / (TP + FP)$
 Recall = $TP / (TP + FN)$
 F1 = $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

Expected Participant Reasoning

Teams should compute accuracy for both guards, see that Guard-B 'wins', and then ask what Guard-B actually does. Reading the matrix, Guard-B never predicts ATTACK at all: it is the lazy model that calls everything normal. Its accuracy comes entirely from the 980 normal events. Once teams reject it, they extract TP, FP and FN for Guard-A and calculate precision, recall and F1. No ML background is needed because every formula is supplied; the insight is in interpreting the data.

Step-by-Step Solution

1. Accuracy: Guard-A = $(16 + 956) / 1000 = 97.2\%$. Guard-B = $(0 + 980) / 1000 = 98.0\%$. NEXUS's claim is arithmetically true, but misleading.
2. Guard-B's recall = $0 / 20 = 0\%$. It catches none of the 20 attacks, and under the stated cost of a missed attack it is useless. Guard-A catches 16 of 20.
3. Trust **Guard-A**.
4. From Guard-A: TP = 16, FP = 24, FN = 4.
5. Precision = $16 / (16 + 24) = 16/40 = 0.40$. Recall = $16 / (16 + 4) = 16/20 = 0.80$.

6. $F1 = 2 \times 0.40 \times 0.80 / (0.40 + 0.80) = 0.64 / 1.20 = 0.5333...$ which rounds to **53%**.

7. Cross-check with $F1 = 2TP / (2TP + FP + FN) = 32 / 60 = 0.5333$. Authorisation code: **A-53**.

Final Answer

A-53

Key Fragment

A-53 (the letter is the trusted guard; the number is its F1 score.)

Concepts Tested

Confusion matrix, TP / FP / FN / TN, accuracy versus recall on imbalanced data, precision, recall, F1, choosing a metric from the real-world cost of errors.

Common Mistakes

- Trusting NEXUS and answering **B-98** (the accuracy trap; the intended wrong answer).
- Answering with Guard-A's accuracy (A-97), recall (A-80) or precision (A-40) instead of F1.
- Submitting 53.3 or 53.33 instead of a whole number.
- Swapping FP and FN when reading the matrix. This produces the same F1 (precision and recall just swap), so it is harmless here.

Hint 1

Compare how many real attacks each guard actually caught, not how many events each one labelled correctly overall.

Hint 2

Guard-B's high accuracy comes almost entirely from the 980 normal events. For the guard that does catch attacks, read TP, FP and FN from the matrix, compute precision and recall first, and only then combine them into F1.

Evaluation Criteria

Accepted answers: A-53 , A53 , A-53% (any capitalisation).

If a team submits A-53.3 or A-53.33 , say 'whole number, please' and let them resubmit without counting it as a wrong attempt.

Any other answer is incorrect, including B-anything, A-97, A-80 and A-40.

LEVEL 3 — The Imposter Processes

Difficulty

Medium

Recommended Time

22 minutes

Technical Domain

Data analysis / pattern recognition

Story Context

With the guard restored, NEXUS scans its own memory and finds that 14 processes are running. Some of them are not NEXUS's. The intruder planted imposter processes by hand and tuned them to keep CPU usage low so that no alarm would fire. NEXUS cannot recognise them by behaviour, but every legitimate process was created by the allocator, and the allocator never improvises.

Challenge Presented to Participants

```
>> MEMORY SCANNER: 14 processes found. Some are IMPOSTERS.  
>> NEXUS's allocator is deterministic. Every LEGITIMATE process was given  
    its memory by ONE exact rule that depends on exactly ONE other column  
    of this table. No rounding, no noise.  
>> The imposters were planted by hand, so they break the rule. They also  
    keep their CPU use low to stay quiet.  
>> Find EVERY imposter.  
>> Quarantine code = the SUM of the PIDs of all imposters.
```

Data / Code / Clues Given to Participants

PID	Process	Threads	Memory (MB)	CPU %
101	auth_svc	4	104	12
102	log_daemon	2	56	3
103	net_probe	6	152	41
104	ml_core	10	248	88

PID	Process	Threads	Memory (MB)	CPU %
105	cache_mgr	3	80	7
106	scheduler	5	128	15
107	shadow_svc	3	200	9
108	io_buffer	8	200	22
109	updater	2	56	2
110	sync_tool	7	184	14
111	kern_helper	4	232	5
112	metrics	6	152	19
113	ghost_proc	9	160	6
114	vault	1	32	4

Expected Participant Reasoning

Teams must work out which column drives memory (CPU is noise; Threads is the driver), discover that the relationship is linear, derive the formula, and then test every row against it. The deception is that imposters are not all 'too big': one is far too large, one is far too small, and one is off by only 8 MB. Teams that only hunt for big or heavy processes will miss some. Different team members can split the rows and cross-check each other.

Step-by-Step Solution

1. CPU % shows no relationship with memory (the busiest process, ml_core at 88%, is legitimate). Sorting by **Threads** exposes the pattern.
2. Legitimate rows sorted by threads: 1 to 32, 2 to 56, 3 to 80, 4 to 104, 5 to 128, 6 to 152, 8 to 200, 10 to 248. Memory rises by exactly 24 MB per extra thread.
3. Rule: **Memory = 24 x Threads + 8**. Check: 2 threads gives 56, 4 gives 104. The rule fits ten rows exactly.
4. Test every row. Four rows break the rule:
 - 107 shadow_svc: 3 threads should be 80, shows 200 (too high)
 - 110 sync_tool: 7 threads should be 176, shows 184 (off by 8)
 - 111 kern_helper: 4 threads should be 104, shows 232 (too high)
 - 113 ghost_proc: 9 threads should be 224, shows 160 (too low)
5. Sum of imposter PIDs: 107 + 110 + 111 + 113 = **441**.

Final Answer

441

Key Fragment

441

Concepts Tested

Exploratory data analysis, ignoring distractor variables, linear relationships, fitting a rule from clean data, outlier detection, systematic verification of every record.

Common Mistakes

- Suspecting the highest-CPU processes (ml_core, net_probe), which are legitimate.
- Flagging only the obviously large rows (107 and 111) and answering 218.
- Missing the too-low row 113 or the subtle row 110: $107 + 111 + 113 = 331$, or $107 + 111 = 218$.
- Fitting a line through the whole table including the imposters and flagging the wrong rows.
- Giving PIDs as a list instead of their sum.

Hint 1

Ignore the CPU column. Sort or group the table by a different column and watch how the legitimate memory values behave.

Hint 2

When sorted by Threads, legitimate memory values climb by the same fixed amount for each extra thread. Find that amount and the starting offset, then check every single row. Imposters can be too high, too low, or only slightly wrong.

Evaluation Criteria

Accepted answer: 441 . Any other number is incorrect.

Organizer shortcut: if a team reports a PID list rather than a sum, the correct set is exactly {107, 110, 111, 113}. A set missing or adding any PID is wrong. A correct list must still be converted to 441 before the answer is accepted.